



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 10: Desarrollo Full-Stack por Capas con Spring Boot y Angular Standalone (VideoRent)

Resumen

Esta práctica guiada consolida los conceptos del **Tema 4.6 (Framework Front-End Angular)** e integra el modelo de desarrollo **Full-Stack desacoplado por capas**. Usted aprenderá a estructurar una solución web profesional dividida en dos módulos independientes dentro de un mismo espacio de trabajo: un **backend RESTful** desarrollado en Java con Spring Boot 3 y un **frontend SPA (Single Page Application)** construido con Angular Standalone.

A lo largo del laboratorio, usted extenderá la plataforma de alquiler de películas **VideoRent**. En la capa backend, organizará la persistencia, los objetos de transferencia de datos (DTOs), la lógica de negocio y los controladores RESTful con soporte para CORS. En la capa frontend, aprenderá a inicializar un proyecto Angular desde cero utilizando **Angular CLI** (`ng new --standalone`), configurar la inyección de dependencias HTTP (`HttpClient`), crear interfaces TypeScript, construir

componentes Standalone con plantillas HTML y CSS, y lograr una comunicación fluida y reactiva en tiempo real entre el servidor y el navegador.

Metadatos del Laboratorio

- **Tiempo estimado:** 4 horas.
- **Herramientas requeridas:** Java 21 (o versión local instalada), Spring Boot 3.x, Node.js 18+, Angular CLI (`@angular/cli`), Maven, Visual Studio Code / IntelliJ IDEA, Navegador Web (Chrome/Firefox).

Metas de Aprendizaje:

Estructurar una solución Full-Stack desacoplada organizando el proyecto en carpetas independientes (`videorent-backend` y `videorent-frontend`).

Implementar la arquitectura de capas en Spring Boot (Entidades JPA, Repositorios, DTOs, Servicios de Negocio, Controladores RESTful y Filtros CORS).

Crear y configurar un nuevo proyecto de Angular Standalone utilizando la consola de comandos de Angular CLI (`ng new videorent-frontend --standalone`) sin SSR y con formato CSS.

Configurar la inyección de dependencias de `HttpClient` en Angular 19/18 mediante `app.config.ts`.

Desarrollar componentes Standalone, modelos de TypeScript y servicios HTTP en Angular para consumir endpoints RESTful e interactuar dinámicamente con el catálogo de alquiler de videos.

Diagnosticar y corregir errores comunes de integración Full-Stack, como bloqueos por políticas CORS (`Access-Control-Allow-Origin`), inyectores HTTP no registrados y conflictos de puertos locales.

Conceptos Clave (El 'Qué')

1. Arquitectura Full-Stack Desacoplada

Una arquitectura desacoplada separa estrictamente las responsabilidades del servidor (backend) y del cliente (frontend). El backend expone servicios RESTful sin importar qué cliente los consuma, mientras que el frontend SPA en Angular gestiona la interfaz de usuario, las vistas y la interacción en el navegador sin provocar recargas globales de página.

2. Capas del Backend (Persistence, DTO, Service, Controller)

- **Dominio/Persistencia (@Entity / JpaRepository):** Define el modelo relacional y la comunicación directa con la base de datos.
- **DTO (Data Transfer Object):** Protege la entidad interna exponiendo únicamente los atributos necesarios hacia el exterior.
- **Servicio (@Service):** Encapsula las reglas de negocio (validar disponibilidad, cambiar estados, calcular totales).
- **Controlador (@RestController):** Atiende peticiones HTTP (GET, POST, PATCH), mapea rutas y aplica políticas CORS.

3. Componentes Standalone de Angular

En versiones modernas de Angular, los componentes Standalone prescinden de los módulos tradicionales (NgModule). Cada componente gestiona sus propias dependencias mediante el decorador `@Component({ standalone: true, imports: [...] })`, simplificando la estructura del proyecto y mejorando la mantenibilidad.

4. Inyección de Dependencias y HttpClient

Angular proporciona un contenedor de inyección de dependencias. Para realizar peticiones HTTP asíncronas (`http.get()`, `http.post()`), se registra `provideHttpClient()` en `app.config.ts` y se inyecta la clase `HttpClient` en servicios Angular `@Injectable()`.

Arquitectura de la Solución Multicapa

Para enseñar a trabajar en capas separadas, la estructura física del proyecto en su entorno de trabajo se organizará de la siguiente forma:

```
videorent-project/
├── videorent-backend/      <-- Proyecto Spring Boot (Java 21)
│   ├── src/main/java/com/videorent/
│   │   ├── model/Video.java
│   │   ├── repository/VideoRepository.java
│   │   ├── dto/
│   │   │   ├── VideoDTO.java
│   │   │   ├── AlquilerVideoDTO.java
│   │   │   └── service/
│   │   │       ├── VideoService.java
│   │   │       └── VideoServiceImpl.java
│   │   └── controller/VideoController.java
│   │   └── config/CorsConfig.java
│   └── src/main/resources/application.properties
└── videorent-frontend/    <-- Proyecto Angular (TypeScript)
    ├── src/app/
    │   ├── models/video.model.ts
    │   ├── services/video.service.ts
    │   ├── components/catalogo-video/
    │   │   ├── catalogo-video.component.ts
    │   │   ├── catalogo-video.component.html
    │   │   └── catalogo-video.component.css
    │   ├── app.component.ts
    │   ├── app.component.html
    │   ├── app.config.ts
    │   └── app.routes.ts
    ├── angular.json
    └── package.json
```

Parte 1: Construcción del Backend RESTful por Capas

(videorent-backend)

Paso 1.1: Entidad de Dominio (Video.java)

Cree la clase `Video.java` en el paquete `com.videorent.model`:

```
package com.videorent.model;

import jakarta.persistence.*;

@Entity
@Table(name = "Video")
public class Video {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String titulo;

    @Column(nullable = false, length = 100)
    private String director;

    @Column(nullable = false, length = 50)
    private String categoria;

    @Column(name = "precio_alquiler", nullable = false)
    private Double precioAlquiler;

    @Column(nullable = false)
    private Boolean disponible;

    public Video() {}

    public Video(String titulo,
                  String director,
                  String categoria,
                  Double precioAlquiler,
                  Boolean disponible) {
        this.titulo = titulo;
        this.director = director;
        this.categoria = categoria;
        this.precioAlquiler = precioAlquiler;
        this.disponible = disponible;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
```

```

    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getDirector() { return director; }
    public void setDirector(String director) {
        this.director = director;
    }

    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public Double getPrecioAlquiler() {
        return precioAlquiler;
    }
    public void setPrecioAlquiler(Double precioAlquiler) {
        this.precioAlquiler = precioAlquiler;
    }

    public Boolean getDisponible() { return disponible; }
    public void setDisponible(Boolean disponible) {
        this.disponible = disponible;
    }
}

```

Paso 1.2: Repositorio JPA (`VideoRepository.java`)

Cree la interfaz `VideoRepository.java` en el paquete `com.videorent.repository`:

```

package com.videorent.repository;

import com.videorent.model.Video;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface VideoRepository extends JpaRepository<Video, Long> {

```

```
List<Video> findByDisponibleTrue();  
}
```

Paso 1.3: Objetos de Transferencia de Datos (VideoDTO.java y AlquilerVideoDTO.java)

Cree las clases DTO en el paquete com.videorent.dto:

VideoDTO.java:

```
package com.videorent.dto;  
  
public class VideoDTO {  
  
    private Long id;  
    private String titulo;  
    private String director;  
    private String categoria;  
    private Double precioAlquiler;  
    private Boolean disponible;  
  
    public VideoDTO() {}  
  
    public VideoDTO(Long id,  
                    String titulo,  
                    String director,  
                    String categoria,  
                    Double precioAlquiler,  
                    Boolean disponible) {  
        this.id = id;  
        this.titulo = titulo;  
        this.director = director;  
        this.categoria = categoria;  
        this.precioAlquiler = precioAlquiler;  
        this.disponible = disponible;  
    }  
  
    public Long getId() { return id; }  
    public void setId(Long id) { this.id = id; }  
  
    public String getTitulo() { return titulo; }
```

```

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getDirector() { return director; }
    public void setDirector(String director) {
        this.director = director;
    }

    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public Double getPrecioAlquiler() {
        return precioAlquiler;
    }
    public void setPrecioAlquiler(Double precioAlquiler) {
        this.precioAlquiler = precioAlquiler;
    }

    public Boolean getDisponible() { return disponible; }
    public void setDisponible(Boolean disponible) {
        this.disponible = disponible;
    }
}

```

AlquilarVideoDTO.java:

```

package com.videorent.dto;

public class AlquilarVideoDTO {

    private String usuario;

    public AlquilarVideoDTO() {}

    public AlquilarVideoDTO(String usuario) {
        this.usuario = usuario;
    }

    public String getUsuario() { return usuario; }
    public void setUsuario(String usuario) {
        this.usuario = usuario;
    }
}

```

```
}
```

Paso 1.4: Capa de Servicio (VideoService.java y VideoServiceImpl.java)

Cree la interfaz VideoService.java en com.videorent.service:

```
package com.videorent.service;

import com.videorent.dto.VideoDTO;
import java.util.List;

public interface VideoService {
    List<VideoDTO> obtenerTodosLosVideos();
    VideoDTO obtenerPorId(Long id);
    VideoDTO alquilarVideo(Long id, String usuario);
}
```

Cree la implementación VideoServiceImpl.java en com.videorent.service:

```
package com.videorent.service;

import com.videorent.dto.VideoDTO;
import com.videorent.model.Video;
import com.videorent.repository.VideoRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class VideoServiceImpl implements VideoService {

    private final VideoRepository videoRepository;

    public VideoServiceImpl(VideoRepository videoRepository) {
        this.videoRepository = videoRepository;
    }

    @Override
    @Transactional(readOnly = true)
```

```

    public List<VideoDTO> obtenerTodosLosVideos() {
        return videoRepository.findAll()
            .stream()
            .map(this::convertirADTO)
            .collect(Collectors.toList());
    }

    @Override
    @Transactional(readOnly = true)
    public VideoDTO obtenerPorId(Long id) {
        Video video = videoRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Video no encontrado con ID: " + id));
        return convertirADTO(video);
    }

    @Override
    @Transactional
    public VideoDTO alquilarVideo(Long id, String usuario) {
        Video video = videoRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Video no encontrado con ID: " + id));

        if (!video.getDisponible()) {
            throw new IllegalStateException(
                "El video '" + video.getTitulo() +
                "' no se encuentra disponible.");
        }

        video.setDisponible(false);
        Video videoActualizado = videoRepository.save(video);
        return convertirADTO(videoActualizado);
    }

    private VideoDTO convertirADTO(Video v) {
        return new VideoDTO(
            v.getId(),
            v.getTitulo(),
            v.getDirector(),
            v.getCategoria(),
            v.getPrecioAlquiler(),
            v.getDisponible()
        );
    }
}

```

Paso 1.5: Capa de Controlador RESTful y Configuración CORS

(VideoController.java)

Cree la clase `VideoController.java` en `com.videorent.controller`:

```
package com.videorent.controller;

import com.videorent.dto.AlquilarVideoDTO;
import com.videorent.dto.VideoDTO;
import com.videorent.service.VideoService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/videos")
@CrossOrigin(origins = "http://localhost:4200")
public class VideoController {

    private final VideoService videoService;

    public VideoController(VideoService videoService) {
        this.videoService = videoService;
    }

    @GetMapping
    public ResponseEntity<List<VideoDTO>> obtenerVideos() {
        List<VideoDTO> lista = videoService.obtenerTodosLosVideos();
        return ResponseEntity.ok(lista);
    }

    @GetMapping("/{id}")
    public ResponseEntity<VideoDTO> obtenerPorId(
        @PathVariable Long id) {
        return ResponseEntity.ok(videoService.obtenerPorId(id));
    }

    @PostMapping("/{id}/alquilar")
    public ResponseEntity<VideoDTO> alquilar(
        @PathVariable Long id,
        @RequestBody AlquilarVideoDTO dto) {
        VideoDTO actualizado = videoService.alquilarVideo(
```

```
        id, dto.getUsuario());  
        return ResponseEntity.ok(actualizado);  
    }  
}
```

Cree el archivo `data.sql` en `src/main/resources/data.sql` para poblar datos semilla:

```
INSERT INTO Video (titulo, director, categoria, precio_alquiler, disponible)  
VALUES ('Inception', 'Christopher Nolan', 'Ciencia Ficción', 3.50, true);  
  
INSERT INTO Video (titulo, director, categoria, precio_alquiler, disponible)  
VALUES ('The Matrix', 'Lana & Lilly Wachowski', 'Acción', 3.00, true);  
  
INSERT INTO Video (titulo, director, categoria, precio_alquiler, disponible)  
VALUES ('Interstellar', 'Christopher Nolan', 'Ciencia Ficción', 4.00, false);  
  
INSERT INTO Video (titulo, director, categoria, precio_alquiler, disponible)  
VALUES ('Pulp Fiction', 'Quentin Tarantino', 'Drama', 2.50, true);
```

Parte 2: Construcción del Frontend Angular (`videorent-frontend`)

A continuación se presentan los pasos oficiales enseñados para crear y configurar una aplicación Angular desde la consola de comandos e inspeccionarla en el IDE.

Paso 2.1: Creación y Configuración del Proyecto Angular

En la consola de comandos (PowerShell o CMD), seleccione su directorio de trabajo: `cmd`
`C:\SW_DEV\Lenguajes2025\Angular>`

Digite el siguiente comando para inicializar el proyecto cliente: `cmd`
`C:\SW_DEV\Lenguajes2025\Angular>ng new videorent-frontend --standalone`

Durante el proceso de inicialización, la consola le solicitará responder las siguientes configuraciones interactivas:

- **Which stylesheet format would you like to use?**

Seleccione: `css`

- **Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)?**

Seleccione: `No`

```
text √ Which stylesheet format would you like to use? CSS √ Do you want to enable
Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No CREATE
videorent-frontend/angular.json (2894 bytes) CREATE videorent-frontend/package.json
(1299 bytes) CREATE videorent-frontend/README.md (1539 bytes) \ Installing packages
(npm)...
```

Explicación técnica de comandos: * `ng`: Comando principal del paquete Angular CLI (@angular/cli). * `ng new`: Subcomando que genera la estructura completa de un nuevo espacio de trabajo de Angular. * `videorent-frontend`: Nombre de la carpeta que contendrá los archivos de desarrollo, dependencias NPM y configuraciones de compilación. * `--standalone`: Parámetro que especifica la arquitectura moderna sin NgModule basada en componentes independientes.

Paso 2.2: Abrir el Proyecto para Codificación en Visual Studio Code

1. Abra **Visual Studio Code**.
2. Seleccione la opción del menú principal: **File > Open Folder...** (o `Ctrl + K, Ctrl + O`).
3. Navegue y seleccione el directorio raíz del proyecto recién creado: `videorent-frontend`.

Examine la estructura en el explorador de archivos (*Explorer*):

```
VIDEORENT-FRONTEND
├── .vscode
├── node_modules
├── public
├── src
│   └── app
│       ├── app.component.css <-- Hoja de estilos del componente principal
│       └── app.component.html <-- Plantilla HTML (vista)
```

```

    app.component.spec.ts
    app.component.ts      <-- Controlador TypeScript (lógica)
    app.config.ts         <-- Configuración de proveedores e inyectores
    app.routes.ts         <-- Tabla de enrutamiento
    index.html
    main.ts
    styles.css
angular.json
package.json             <-- Lista de dependencias asociadas y scripts
tsconfig.json

```

Estructura Interna de un Componente Angular: * **CSS** (`app.component.css`) y **HTML** (`app.component.html`): Representan la **Vista (View)**. * **TS** (`app.component.ts`): Representa el **Controlador (Controller)** en TypeScript que gestiona el estado y la interactividad. * **package.json**: Lista de dependencias de software asociadas al proyecto Angular y sus versiones exactas.

Paso 2.3: Iniciar las Herramientas de Desarrollo de Angular (`ng serve`)

En el terminal integrado de VS Code o en la consola de comandos, ubíquese dentro del directorio del proyecto frontend: `cmd`
`C:\SW_DEV\Lenguajes2025\Angular\videorent-frontend>ng serve`

Espere a que Angular CLI termine el proceso de compilación inicial en memoria:

```

Initial chunk files | Names | Raw size
polyfills.js | polyfills | 90.20 kB
main.js | main | 23.11 kB
styles.css | styles | 95 bytes

```

Application bundle generation complete. [4.925 seconds]

Watch mode enabled. Watching for file changes... → Local: <http://localhost:4200/>

1. Abra su navegador web y navegue a la dirección URL: `http://localhost:4200/`.
2. Verifique que la pantalla predeterminada de bienvenida de Angular ("Hello, videorent-frontend") se renderice correctamente.

Paso 2.4: Configuración de `HttpClient` en `app.config.ts`

Para consumir la API REST del backend `videorent-backend`, debe registrar el proveedor de clientes HTTP en la configuración global de la aplicación.

Abra el archivo `src/app/app.config.ts` y modifíquelo para incluir `provideHttpClient()`:

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient()
  ]
};
```

Paso 2.5: Creación del Modelo de Datos (`video.model.ts`)

Cree la carpeta `src/app/models/` y dentro de ella el archivo `video.model.ts`:

```
export interface Video {
  id: number;
  titulo: string;
  director: string;
  categoria: string;
  precioAlquiler: number;
  disponible: boolean;
}

export interface AlquilerVideoPayload {
  usuario: string;
}
```

Paso 2.6: Generación e Implementación del Servicio Angular (`VideoService`)

En la consola de comandos, dentro del directorio `videorent-frontend`, ejecute la herramienta CLI para generar el servicio:

```
ng generate service services/video
```

Abra el archivo generado `src/app/services/video.service.ts` y escriba la lógica de comunicación HTTP:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Video, AlquilerVideoPayload } from '../models/video.model';

@Injectable({
  providedIn: 'root'
})
export class VideoService {
  private apiUrl = 'http://localhost:8080/api/videos';

  constructor(private http: HttpClient) {}

  obtenerCatalogos(): Observable<Video[]> {
    return this.http.get<Video[]>(this.apiUrl);
  }

  alquilarVideo(id: number, usuario: string): Observable<Video> {
    const payload: AlquilerVideoPayload = { usuario };
    return this.http.post<Video>(
      `${this.apiUrl}/${id}/alquilar`,
      payload
    );
  }
}
```

Paso 2.7: Creación del Componente de Catálogo (`CatalogoVideoComponent`)

Genere el componente de catálogo utilizando Angular CLI:

```
ng generate component components/catalogo-video
```

1. Controlador TypeScript (catalogo-video.component.ts)

Abra src/app/components/catalogo-video/catalogo-video.component.ts:

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Video } from '../../../models/video.model';
import { VideoService } from '../../../services/video.service';

@Component({
  selector: 'app-catalogo-video',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './catalogo-video.component.html',
  styleUrls: ['./catalogo-video.component.css']
})
export class CatalogoVideoComponent implements OnInit {
  videos: Video[] = [];
  cargando: boolean = true;
  mensajeRespuesta: string = '';
  errorMensaje: string = '';
  nombreUsuario: string = 'EstudianteUCR';

  constructor(private videoService: VideoService) {}

  ngOnInit(): void {
    this.cargarCatalogo();
  }

  cargarCatalogo(): void {
    this.cargando = true;
    this.errorMensaje = '';
    this.videoService.obtenerCatalogos().subscribe({
      next: (data) => {
        this.videos = data;
        this.cargando = false;
      },
      error: (err) => {
        console.error('Error al cargar catálogo:', err);
        this.errorMensaje = 'No se pudo conectar con el servidor backend.';
        this.cargando = false;
      }
    });
  }
}
```

```

alquilar(video: Video): void {
  if (!video.disponible) return;

  this.videoService.alquilarVideo(video.id, this.nombreUsuario).subscribe({
    next: (videoActualizado) => {
      video.disponible = videoActualizado.disponible;
      this.mensajeRespuesta = `¡Éxito! Ha alquilado "${video.titulo}".`;
      setTimeout(() => this.mensajeRespuesta = '', 4000);
    },
    error: (err) => {
      console.error('Error al alquilar:', err);
      this.errorMensaje = err.error?.message || 'Error al procesar alquiler.';
      setTimeout(() => this.errorMensaje = '', 4000);
    }
  });
}
}

```

2. Plantilla HTML (catalogo-video.component.html)

Abra `src/app/components/catalogo-video/catalogo-video.component.html`:

```

<div class="container">
  <header class="header">
    <h1>■ VideoRent - Catálogo de Películas</h1>
    <p>Plataforma de alquiler de videos en línea (Full-Stack Angular + Spring Boot)</p>
  </header>

  <div class="user-panel">
    <label for="usuarioInput">Usuario activo:</label>
    <input
      id="usuarioInput"
      type="text"
      [(ngModel)]="nombreUsuario"
      placeholder="Nombre de usuario"
    />
    <button class="btn-secondary" (click)="cargarCatalogo()">
      ■ Recargar Catálogo
    </button>
  </div>

  @if (mensajeRespuesta) {
    <div class="alert alert-success">
      {{ mensajeRespuesta }}
    </div>
  }

```

```

    </div>
  }
}

@if (errorMensaje) {
  <div class="alert alert-danger">
    {{ errorMensaje }}
  </div>
}

@if (cargando) {
  <div class="loading">
    <p>Cargando catálogo de películas desde backend...</p>
  </div>
} @else {
  <div class="grid-catalog">
    @for (video of videos; track video.id) {
      <div class="card" [class.card-disabled]="!video.disponible">
        <div class="card-header">
          <h3>{{ video.titulo }}</h3>
          <span class="badge" [class.badge-available]="video.disponible" [class.badge-rented]="!video.disponible">
            {{ video.disponible ? 'Disponible' : 'Alquilado' }}
          </span>
        </div>
        <div class="card-body">
          <p><strong>Director:</strong> {{ video.director }}</p>
          <p><strong>Categoría:</strong> {{ video.categoria }}</p>
          <p class="price"><strong>Precio:</strong> ${{ video.precioAlquiler.toFixed(2) }}</p>
        </div>
        <div class="card-footer">
          <button
            class="btn-primary"
            [disabled]="!video.disponible"
            (click)="alquilar(video)">
            {{ video.disponible ? 'Alquilar Video' : 'No Disponible' }}
          </button>
        </div>
      </div>
    } @empty {
      <p>No se encontraron películas en el catálogo.</p>
    }
  </div>
}
</div>

```

3. Hoja de Estilos CSS (catalogo-video.component.css)

Abra src/app/components/catalogo-video/catalogo-video.component.css:

```
:host {  
  display: block;  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  color: #1e293b;  
  background-color: #f8fafc;  
  min-height: 100vh;  
  padding: 20px;  
}  
  
.container {  
  max-width: 1000px;  
  margin: 0 auto;  
}  
  
.header {  
  text-align: center;  
  margin-bottom: 25px;  
}  
  
.header h1 {  
  color: #0f172a;  
  margin-bottom: 5px;  
}  
  
.user-panel {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  background-color: #ffffff;  
  padding: 15px;  
  border-radius: 8px;  
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.05);  
  margin-bottom: 20px;  
}  
  
.user-panel input {  
  padding: 8px 12px;  
  border: 1px solid #cbd5e1;  
  border-radius: 4px;  
  font-size: 14px;  
}  
  
.alert {  
  padding: 12px 16px;
```

```

border-radius: 6px;
margin-bottom: 20px;
font-weight: 600;
}

.alert-success {
background-color: #dcfce7;
color: #166534;
border: 1px solid #bbf7d0;
}

.alert-danger {
background-color: #fee2e2;
color: #991b1b;
border: 1px solid #fecaca;
}

.grid-catalog {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
gap: 20px;
}

.card {
background: #ffffff;
border-radius: 8px;
box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
display: flex;
flex-direction: column;
justify-content: space-between;
border: 1px solid #e2e8f0;
transition: transform 0.2s ease;
}

.card:hover {
transform: translateY(-4px);
}

.card-disabled {
opacity: 0.75;
background-color: #f1f5f9;
}

.card-header {
padding: 15px;
border-bottom: 1px solid #f1f5f9;

```

```

display: flex;
justify-content: space-between;
align-items: center;
}

.card-header h3 {
margin: 0;
font-size: 18px;
color: #1e293b;
}

.badge {
padding: 4px 8px;
border-radius: 12px;
font-size: 12px;
font-weight: bold;
}

.badge-available {
background-color: #dcfce7;
color: #15803d;
}

.badge-rented {
background-color: #fee2e2;
color: #b91c1c;
}

.card-body {
padding: 15px;
}

.card-body p {
margin: 6px 0;
font-size: 14px;
}

.price {
color: #2563eb;
font-size: 16px !important;
}

.card-footer {
padding: 15px;
border-top: 1px solid #f1f5f9;
}

```

```

.btn-primary {
  width: 100%;
  padding: 10px;
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
}

.btn-primary:hover:not(:disabled) {
  background-color: #1d4ed8;
}

.btn-primary:disabled {
  background-color: #94a3b8;
  cursor: not-allowed;
}

.btn-secondary {
  padding: 8px 14px;
  background-color: #64748b;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

```

Paso 2.8: Montaje del Componente en AppComponent

Abra el controlador principal `src/app/app.component.ts` e importe `CatalogoVideoComponent`:

```

import { Component } from '@angular/core';
import { CatalogoVideoComponent } from '../components/catalogo-video/catalogo-video.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CatalogoVideoComponent],
  templateUrl: './app.component.html',

```

```
styleUrl: './app.component.css'
}))
export class AppComponent {
  title = 'videorent-frontend';
}
```

Abra la plantilla principal `src/app/app.component.html` y reemplace todo su contenido con:

```
<app-catalogo-video></app-catalogo-video>
```

Parte 3: Verificación e Integración Full-Stack

Siga el procedimiento para probar la integración completa entre ambas capas:

1. Iniciar el Backend Spring Boot:

2. Abra la terminal en la carpeta `videorent-backend` y ejecute: `cmd mvnw spring-boot:run`

Verifique en los logs que el servidor esté escuchando en el puerto `8080`.

Iniciar el Frontend Angular:

5. Abra otra terminal en la carpeta `videorent-frontend` y ejecute: `cmd ng serve`

Abra el navegador en `http://localhost:4200/`.

Prueba Interactiva:

8. Observe cómo Angular realiza automáticamente la petición `GET /api/videos` y renderiza las películas cargadas desde la base de datos backend.
9. Seleccione un video en estado **Disponible** y haga clic en **Alquilar Video**.
10. Verifique que la aplicación envíe la petición `POST /api/videos/{id}/alquilar`, el backend actualice el estado a `disponible = false` en la base de datos, y Angular actualice instantáneamente el badge a **Alquilado** (rojo) y deshabilite el botón sin recargar la página.
-

Parte 4: Guía de Depuración de Errores Comunes

Error 1: Bloqueo por Política CORS (`Access-Control-Allow-Origin`)

- **Síntoma:** En la consola del navegador aparece `Access to XMLHttpRequest at 'http://localhost:8080/api/videos' from origin 'http://localhost:4200' has been blocked by CORS policy.`
- **Causa:** El backend Spring Boot no está autorizando peticiones provenientes del origen `http://localhost:4200`.
- **Solución:** Verifique que la anotación `@CrossOrigin(origins = "http://localhost:4200")` esté presente sobre el controlador `VideoController.java`.

Error 2: `NullInjectorError: No provider for HttpClient!`

- **Síntoma:** En la consola del navegador se muestra el error `NullInjectorError: R3InjectorError(Standalone[_AppComponent])[... -> HttpClient -> HttpClient]`.
- **Causa:** No se configuró el proveedor de peticiones HTTP en el archivo de configuración global de Angular.
- **Solución:** Incluya `provideHttpClient()` dentro del arreglo `providers` en `src/app/app.config.ts`.

Error 3: Puerto 4200 u 8080 Ocupado (`Port 4200 is already in use`)

- **Síntoma:** Angular CLI o Spring Boot rehusán iniciar indicando que el puerto ya está en uso por otro proceso.
- **Solución:** En Angular, puede especificar un puerto alternativo ejecutando `ng serve --port 4201`. Para Spring Boot, cambie el puerto en `application.properties` agregando `server.port=8081`.

Parte 5: Reto Autónomo Evaluado

Para consolidar lo aprendido en esta práctica guiada, implemente los siguientes requerimientos adicionales de forma autónoma:

1. **Funcionalidad de Devolución de Película:**

2. **Backend:** Agregue un endpoint `@PostMapping("/{id}/devolver")` en `VideoController.java` y su método correspondiente en `VideoService` para cambiar el estado del video a `disponible = true`.

Frontend: Modifique `VideoService` en Angular para consumir el endpoint de devolución. Agregue un botón secundario **"Devolver Video"** en `catalogo-video.component.html` que solo esté visible cuando el video se encuentre en estado **Alquilado** (`!video.disponible`).

Filtro Dinámico de Películas:

5. Agregue un campo de texto `<input type="text">` en `catalogo-video.component.html` para filtrar el catálogo en tiempo real por el título de la película utilizando TypeScript.

Rúbrica de Evaluación (100 Puntos)

Criterio	Puntaje	Descripción
Backend RESTful por Capas	30 pts	Implementación correcta de entidades, DTOs, servicios, repositorios y controladores RESTful con soporte CORS.
Creación e Integración Angular	30 pts	Generación correcta del proyecto Angular Standalone, configuración de <code>HttpClient</code> y servicios inyectables.

Componente e Interfaz SPA	25 pts	Desarrollo del componente con data binding, control de flujo (@if, @for), diseño responsive y actualización de estados.
Reto Autónomo (Devolución y Filtro)	15 pts	Implementación exitosa de los endpoints y componentes de devolución de películas y filtro dinámico.